
Collaborative To-Do List Application

by An Phan and Vatsalkumar Mukeshkumar Dholakiya (Group 9)
University of the Cumberland
Github: <https://github.com/anphan7/MSCS-632-Residency>

Task

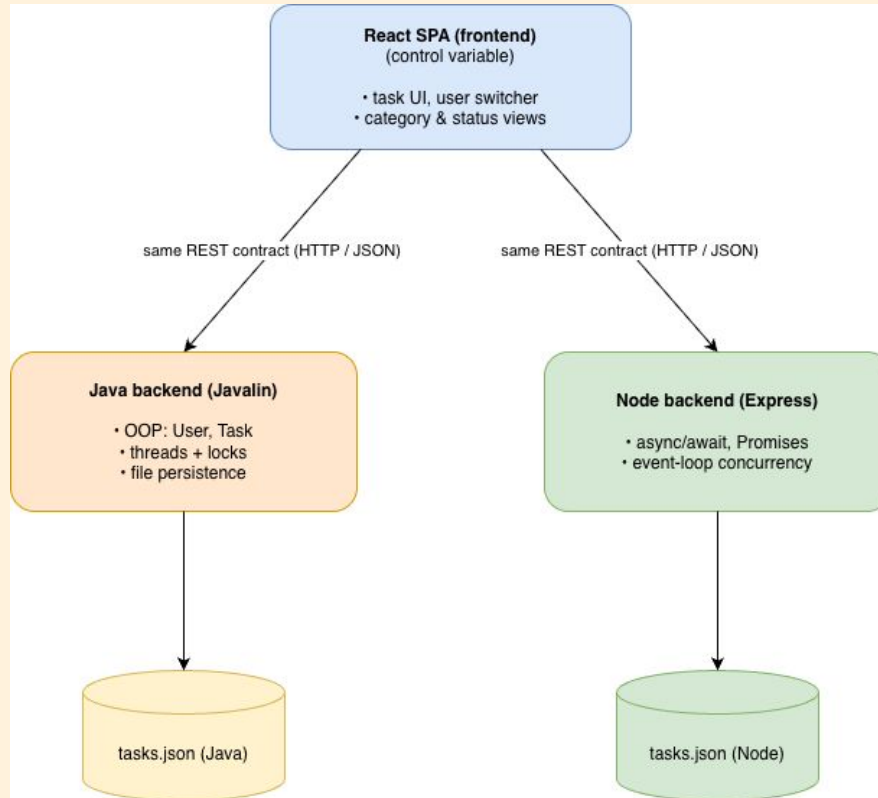
An Phan

- Architecture design
- Developer (Java + Frontend)
- API

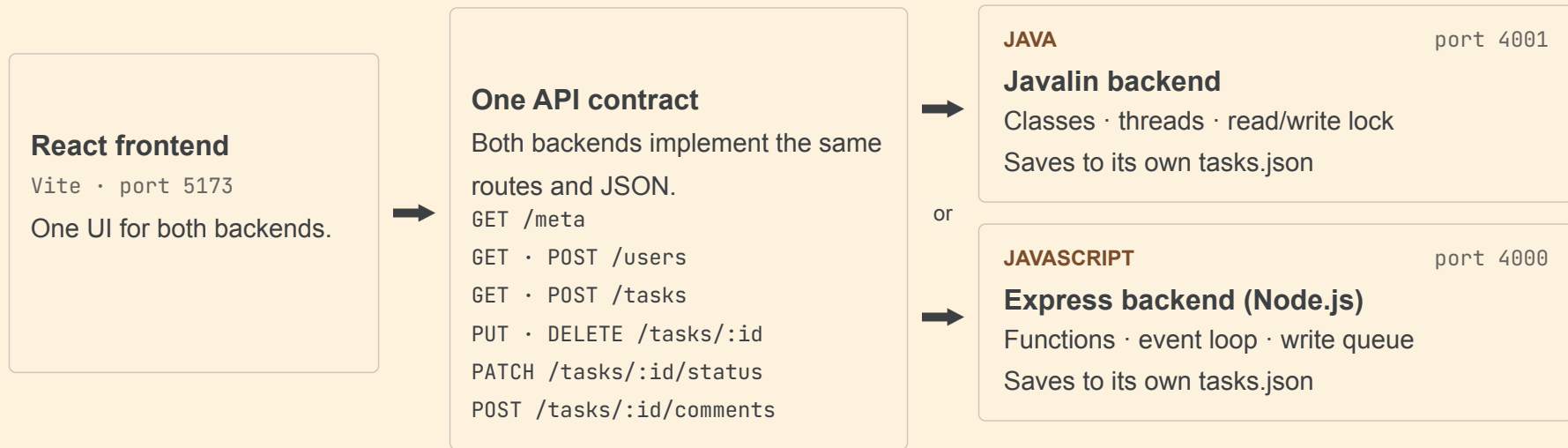
Vatsalkumar Mukeshkumar Dholakiya

- Developer (Javascript)
- Report documents

Architecture Diagram



API Architecture



Application Features

- Multi-users / shared team board
- User switcher, add new user
- Create, edit, delete and assign tickets
- Label ticket for quick search
- 5 statuses (Open, In Progress, Completed, Deprecated, Need Requirement)
- Filter by status, tag, assignee
- Comments per ticket

Live Demo

Classes vs. functions

Java

class · static types

```
class Task {  
    String id  
    String title  
    String status = "Open"  
}  
  
service = new TaskService(repository)
```

JavaScript

factory · closure · plain object

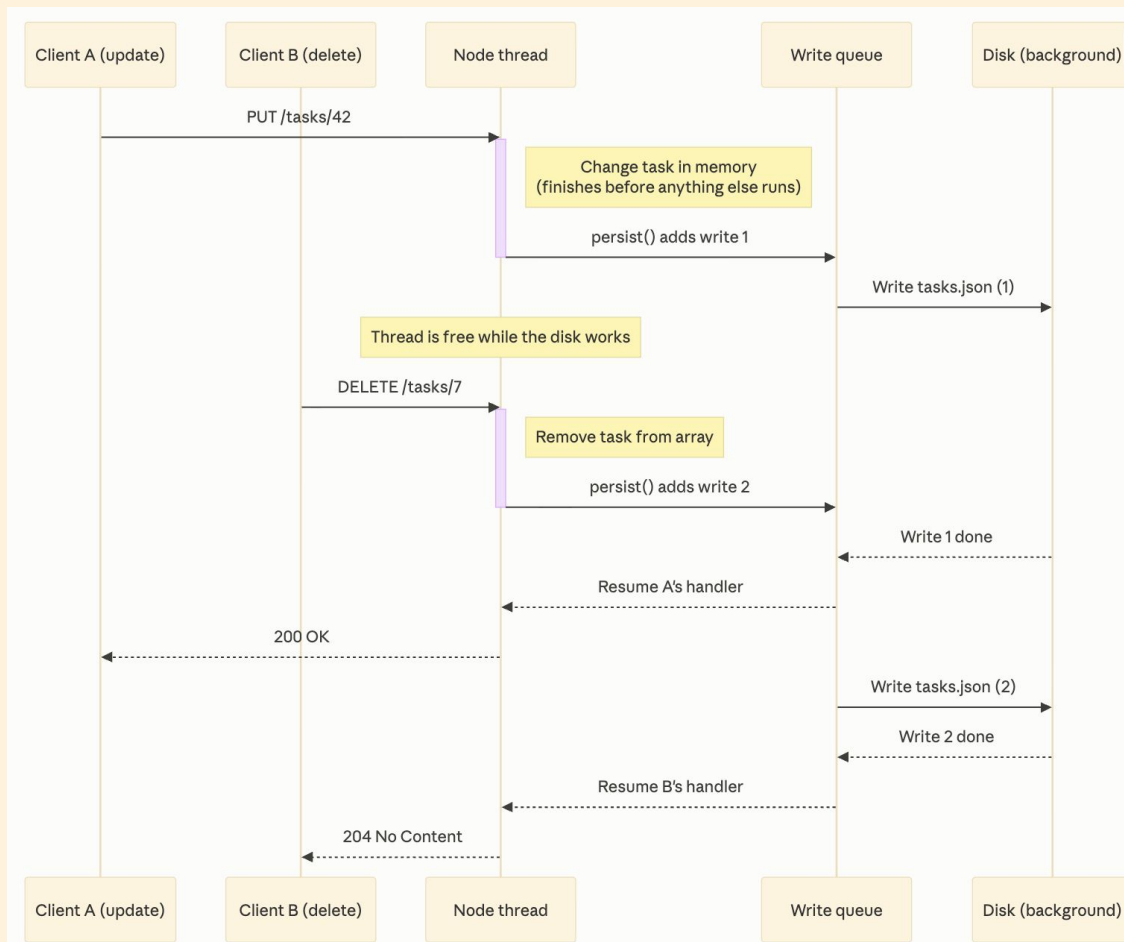
```
function createService(store) {  
    function addTask(input) {  
        task = { id, title, status: "Open" }  
        store.save(task)  
    }  
    return { addTask }  
}
```

Syntax	Java	JavaScript
Routes	ctx -> { ... }	async (req, res) => { ... }
Errors	throw IllegalArgumentException	throw ValidationError
Defaults	s == null ? "Open" : s	s ?? "Open"

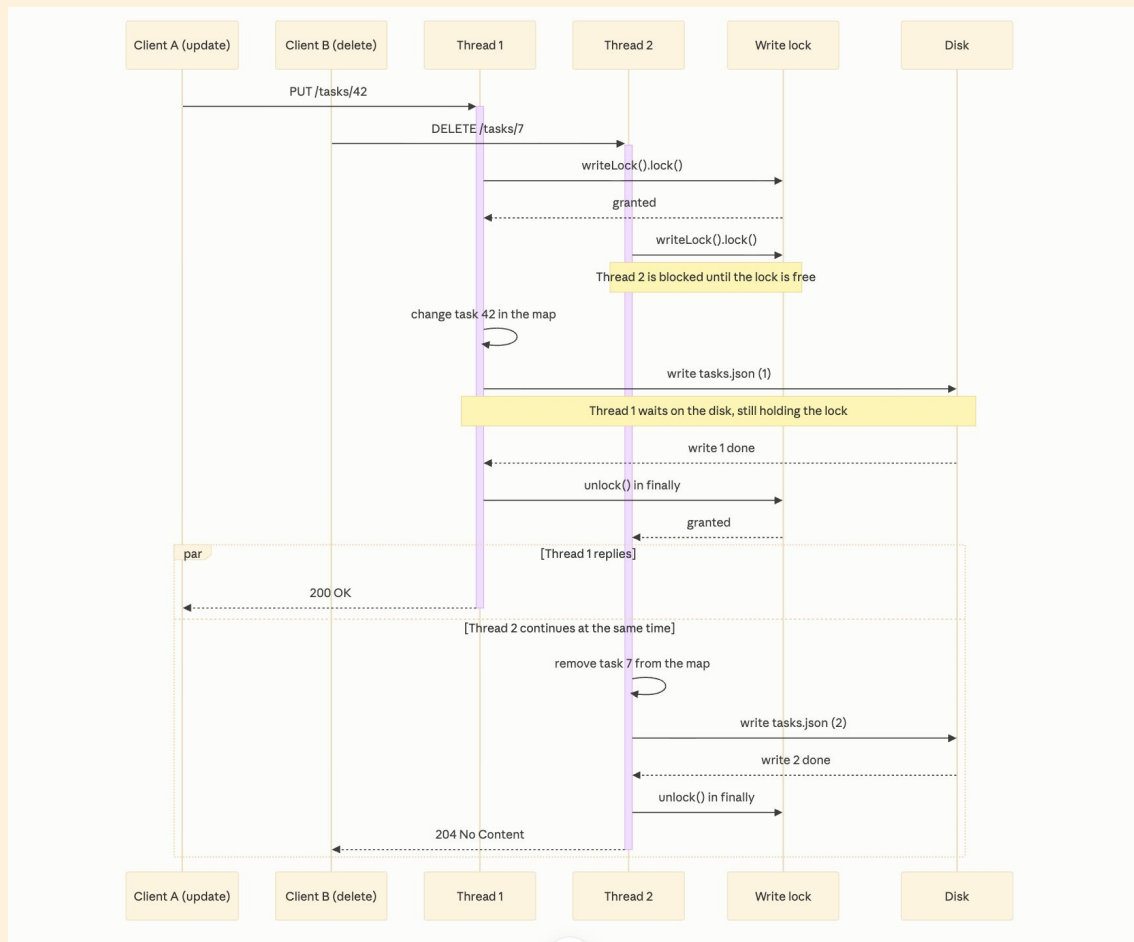
Features Differences

Features	Java	Javascript
Saving a task	Locks the data with a write lock	Queues the file write
Creating a task (save as JSON)	Builds a typed <i>Task</i> object and serializes with Jackson	Builds a plain object and uses native <i>JSON.stringify</i>
Incoming JSON	Jackson binds the JSON to a typed object	Already a parsed object - no binding needed
Storing tasks/users	A typed <i>LinkedHashMap<String, Task></i> + <i>List<User></i> , guarded by the lock.	A plain object <i>{ users: [], tasks: [] }</i> held in a closure.
Editing a task	Runs it inside the write lock	Finds the object and mutates it directly
Filtering tasks	Iterates the map values with explicit if checks under a read lock.	One <i>array.filter()</i> with short-circuit conditions

JavaScript single-threaded



Java multi-threaded



Thank you